Topic: Software packages for quantum computing

What we will cover?

Stage	Lesson	Topics
Stage 1: Understand QML - concepts, applications, two models	1 What is QML? (today)	Taxonomy · Research goals · Tools
	2 Quantum kernel methods	QKM / SVM + RBF kernel
	3 Quantum neural networks	QNN / Neural networks · Wrap-up
Stage 2: Idea to software - research question → paper → package	4 From idea to paper	Research question - First paper
	5 From paper to package	Qmes

What is QML, and what tools do we use?

Lesson 1 · Quantum machine learning

Outline

1. What is QML
2. Taxonomy
3. Research goals
4. Tools

1. Machine learning today

- What ML does: learn patterns from data
  - ❑ Instead of hand-written rules
  - ❑ Images, speech, spam, finance
- What powers it: CPU → GPU
  - ❑ The engine of AI and big data
  - ❑ Data and compute keep growing
- Where it is heading: bigger models
  - ❑ Rising training cost
  - ❑ Moore's law slowing down?
- Meanwhile, in physics: quantum computers, born for a different reason

1. Quantum computers

- Built to simulate physics, not to do machine learning
- Early 1980s · Feynman's idea
  - ❑ Simulate quantum systems with quantum systems
  - ❑ N qubits need 2^N numbers on a classical computer
- 1990s · Beyond simulation
  - ❑ Shor: factoring in polynomial time
  - ❑ Grover: quadratic speedup for unstructured search
- 2019 → · Hardware today
  - ❑ Quantum supremacy on one chosen task
  - ❑ Tens to hundreds of qubits, still noisy
- Note: the most famous result, factoring, has nothing to do with AI at first sight

1. Quantum machine learning

- Machine learning + quantum computing → Quantum machine learning (QML)
  - ❑ Common ground: linear algebra (vectors, matrices, Hilbert space)
- Era 1 · fault-tolerant algorithms
  - ❑ HHL, quantum PCA, quantum SVM
  - ❑ Needs error-corrected hardware
- Era 2 · from ~2018: models for noisy devices
  - ❑ Quantum kernels, quantum neural networks
  - ❑ Lessons 2–3
- QML: running a learning algorithm on a quantum computer for a specific task, hoping for an advantage over classical

2. QML taxonomy?

	Classical algorithm	Quantum algorithm
Classical data	**Classical ML** Images, speech, spam, finance In daily use	**Quantum models on classical data (this course)** Classification, clustering, recommendation, finance Small trials, no known advantage
Quantum data	ML for quantum experiments Phases of matter, state reconstruction, ground states Applied in physics	Quantum processing of quantum data Learning directly from quantum experiments Proof of principle, up to 40 qubits

3. Two research goals

A · Quantum advantage

- ❑ Can quantum do what classical cannot?
- ❑ Now, briefly

B · Quantum learning

- ❑ How do QML models work?
- ❑ Lessons 2–3, in detail

3. Goal A: quantum advantage

Claim	Promise	Conditions
Runtime speedup	Linear systems, PCA, SVM in ~log N instead of N	Sparse, well-conditioned data; error correction; data loading; only one number out
Classical hardness	Quantum kernels that classical computers cannot compute	Proven only on purpose-built problems; error correction; not yet for images or text
Quantum data	Exponentially fewer experiments	Quantum memory; proof of principle so far
Counter-argument: dequantization	Classical algorithm, same data access, nearly as good	The exponential gap disappears

→ It always depends on the data, the input model, and the classical baseline.

3. Why goal A is hard for ML

	Problems studied in quantum computing	Problems solved by machine learning
Classical performance	Weak: problems chosen to be hard	Strong: industrial scale
Input size	Small: few qubits, short bit strings	Very large
Structure	Highly structured	Messy, real-world
Evaluation	Asymptotic complexity	Empirical benchmarks

→ Goal B: understand how QML models work, without having to beat classical

4. Three libraries, three purposes

	PennyLane	Qiskit	Cirq
Built for	Differentiable programming: a circuit as a layer in an ML model	Build circuits, optimize them for a device, run on hardware	Fine-grained control of circuits on today's noisy devices
Gradients & ML	Built in, connects to PyTorch / JAX	Separate library, not in the core	Not the focus
Runs on	Many simulators and hardware via plugins	Hardware via vendor runtimes and providers	Built-in simulators, noise models, Google chips

4. Summary: from taxonomy to tools

1 Taxonomy: where does QML live?	2 Objective: what do researchers want?	3 Platform: how do we build it?
❑ Classical / quantum data × classical / quantum algorithm ❑ This course: quantum models on classical data ❑ Context only: classical ML, ML for quantum experiments, quantum processing of quantum data	❑ A · Quantum advantage: can quantum beat classical? ❑ Runtime speedup · classical hardness · quantum data ❑ Counter-argument: dequantization ❑ B · Quantum learning: how do QML models work? ❑ QKM (Lesson 2) · QNN (Lesson 3)	❑ Qiskit: build circuits, compile, run on hardware ❑ Cirq: fine-grained control of noisy devices ❑ PennyLane: used in this course ❑ Built-in gradients · circuits as Python functions · PyTorch / JAX interfaces

Quantum kernel methods

What does the quantum computer do inside an SVM?

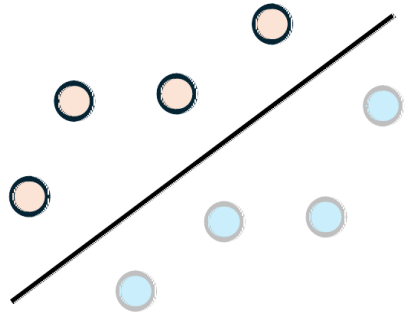
Lesson 2 · Quantum machine learning

Outline

1. SVM & kernels
2. Quantum kernel
3. Cost & failure

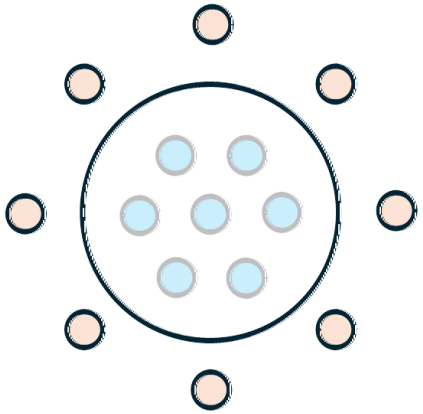
1. Kernel methods: the original idea

(a)



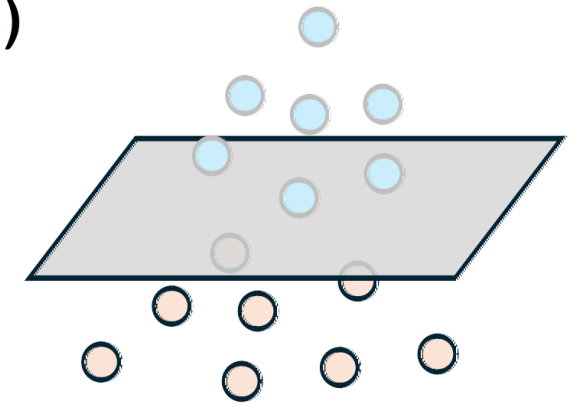
Separable by a line

(b)



Not separable by a line

(c)



Higher dimension: separable by a plane

Map the data to a higher dimension, then separate it with a hyperplane

1. Feature map and kernel

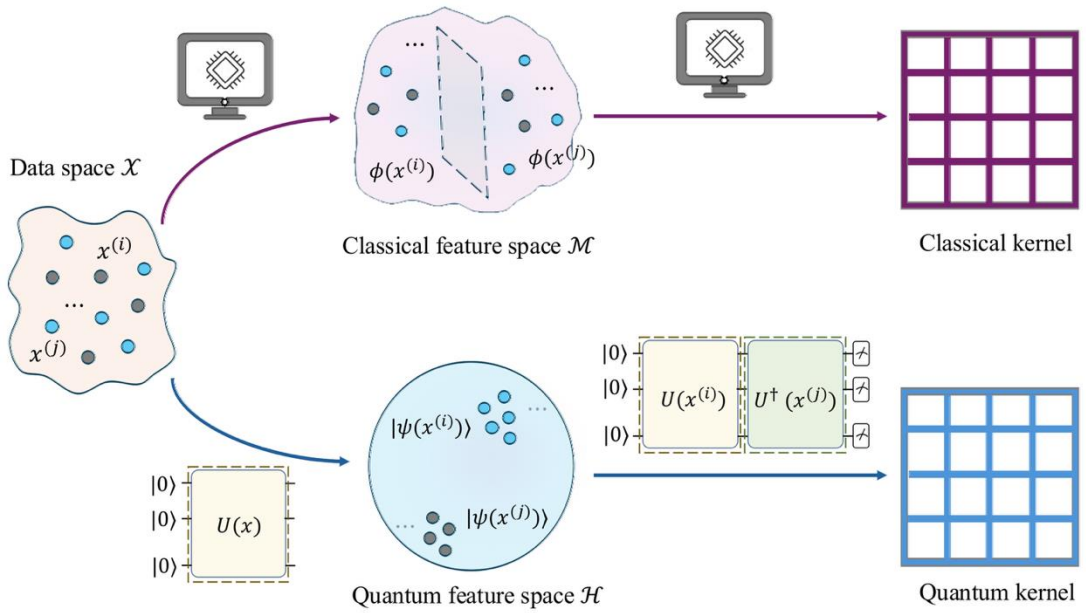
- Feature map
  - $x \in \mathbb{R}^d \rightarrow \varphi(x) \in \mathbb{R}^D, \quad D \geq d$
  - $\varphi(x)$: feature vector
- Kernel
  - $k(x, x') = \langle \varphi(x), \varphi(x') \rangle$
  - Similarity of two points
  - Computed without writing $\varphi(x)$: kernel trick

- RBF kernel: φ hidden, k by formula
  - $k(x, x') = e^{-\gamma \|x - x'\|^2}$
  - $\varphi(x)$ infinite-dimensional, never written down

1. Support vector machine

- Max-margin hyperplane: separates the two classes, as far as possible from both
- Support vectors: the points closest to the hyperplane
- Non-linear data: use a kernel, separate in feature space
- SVM only needs the numbers k

2. Quantum kernel



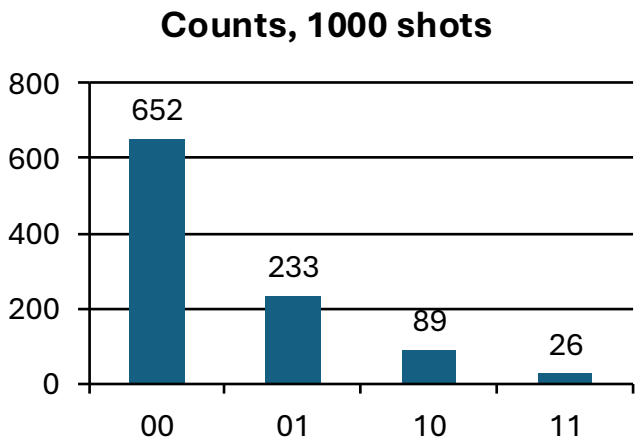
- Feature map
  - $|\psi(x)\rangle = U(x)|0\rangle^{\otimes n}$
  - $U(x)$: data-dependent circuit on n qubits
- Kernel
  - $k(x, x') = |\langle\psi(x)|\psi(x')\rangle|^2$
  - 1: same state · 0: very different
- Only the feature map changes: $\varphi(x) \rightarrow |\psi(x)\rangle$

2. How to calculate K (Gram matrix)

Quantum hardware

One circuit per pair, repeated many times

Circuit: $|00\rangle \rightarrow U(x_1) \rightarrow U^{\dagger}(x_2) \rightarrow$ measure



$K_{12} \approx 652/1000 = 0.652$

2 qubits, toy encoding RX + CNOT, $x_1 = (0.3, 1.2)$, $x_2 = (1.0, 2.3)$

Same K: 0.652 from 1000 shots, 0.641 exact

Simulator (lab)

Simulate each point once, keep the state in memory

$|\psi(x_1)\rangle = (0.816,\ -0.558i,\ -0.084,\ -0.123i)$

$|\psi(x_2)\rangle = (0.358,\ -0.801i,\ -0.438,\ -0.196i)$

$2^n = 4$ numbers per point

$K_{12} = |\langle\psi(x_1)|\psi(x_2)\rangle|^2 = 0.641$

2. QKM = two blocks

- Train data: pairs (x_i, x_j) → quantum block
- **Quantum**: compute K (fixed circuit, nothing trained · hardware or simulator)
  1. Feature map: $|\psi(x)\rangle = U(x)|0\rangle^{\otimes n}$
  2. Kernel evaluation: $K_{ij} = |\langle\psi(x_i)|\psi(x_j)\rangle|^2$
  3. Post-processing: all pairs → matrix K
- **Classical**: SVM
  - ❑Train: $K \rightarrow \alpha, b$ (trained here)
  - ❑Predict: $\text{sign}(\Sigma_i\, \alpha_i\, y_i\, k(x_i, x) + b)$
- Prediction: new x → quantum block gives $k(x_i, x)$ → Predict

2. Training = SVM dual

- Train
  - $\max_{\alpha} \ \Sigma_i \alpha_i - \frac{1}{2} \Sigma_{i,j} \alpha_i \alpha_j y_i y_j \boldsymbol{K_{ij}}$
  - $0 \le \alpha_i \le C, \quad \Sigma_i \alpha_i y_i = 0$
- Predict
  - $\hat{y}(x) = \text{sign}(\Sigma_i \alpha_i y_i \boldsymbol{k(x_i, x)} + b)$
- K_{ij} from
  - Classical (RBF): formula $\exp(-\gamma \|x_i - x_j\|^2)$
  - Quantum: circuit $|\langle \psi(x_i) | \psi(x_j) \rangle|^2$
- Data enters only through K

2. What this means

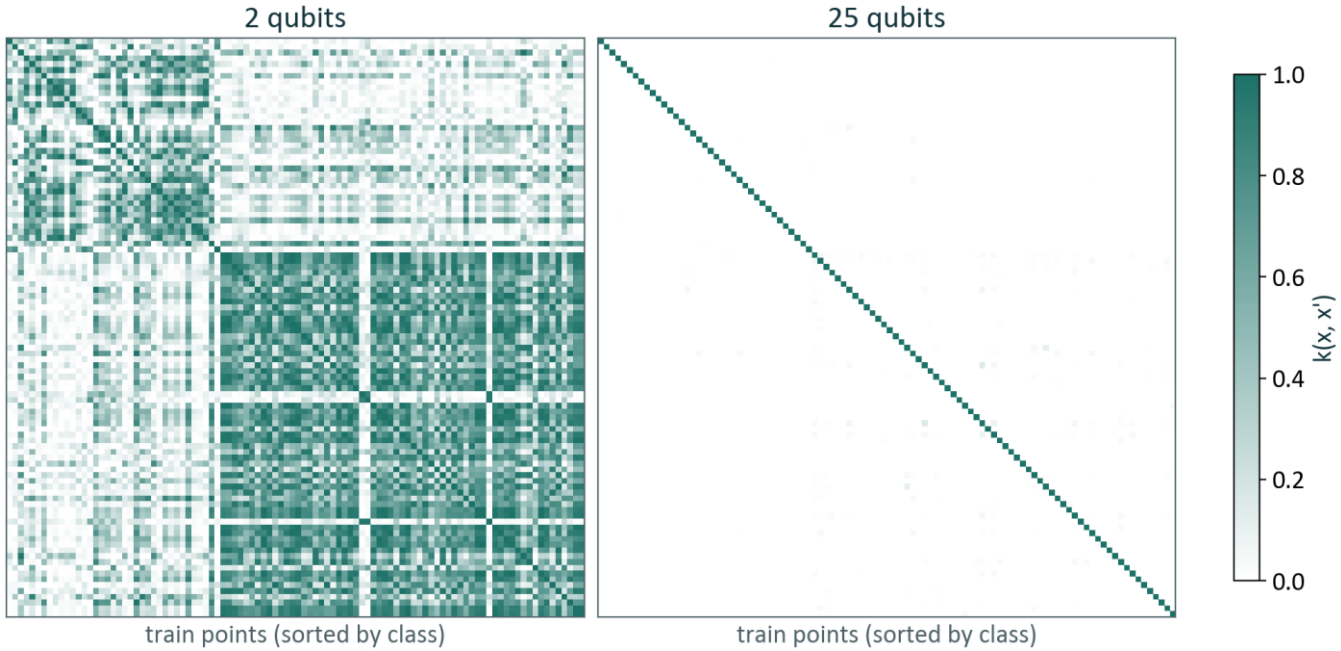
- Quantum: only K
  - ❑Optimization runs on a classical computer
  - ❑SVC(kernel="precomputed")
- Convex
  - ❑Global optimum
  - ❑No circuit gradients
- Quality of K
  - ❑Model quality ≈ quality of K ≈ **choice of encoding**

3. SVM + RBF vs QKMs

	SVM + RBF	QKM
Already seen		
Feature map	Hidden, infinite-dim	Circuit $U(x) \to \|\psi(x)\rangle$, 2^n amplitudes
Kernel	Formula $e^{-\gamma\|x-x'\|^2}$	Run circuit
Optimization, prediction	SVM dual, convex	Identical
What follows		
Hyperparameters	C, γ	C, encoding, data scale
Cost of K	$N(N-1)/2$ formula evaluations	$N(N-1)/2$ circuits × shots
Failure	γ too large → $K \approx I$	$K \approx I$ (next)

3. Exponential concentration: $K \approx I$

1. More qubits → $k(x, x') \to 0$ for $x \neq x'$: $K \approx I$
2. Product encoding: k = product of n numbers < 1
3. Hardware: 0…0 almost never seen: measured $K = I$ → same prediction for every input



Breast cancer, 100 train points, product-RX encoding, exact K

3. Three sources of concentration

- Expressivity: encoding covers too much of state space → states nearly orthogonal
- Global measurement: all n qubits measured at once; even without entanglement
- Noise: hardware noise, worse with deeper circuits
- Fix: keep k from getting tiny
  - ❑Simpler encoding
  - ❑Smaller data scale
  - ❑Encoding built on data structure

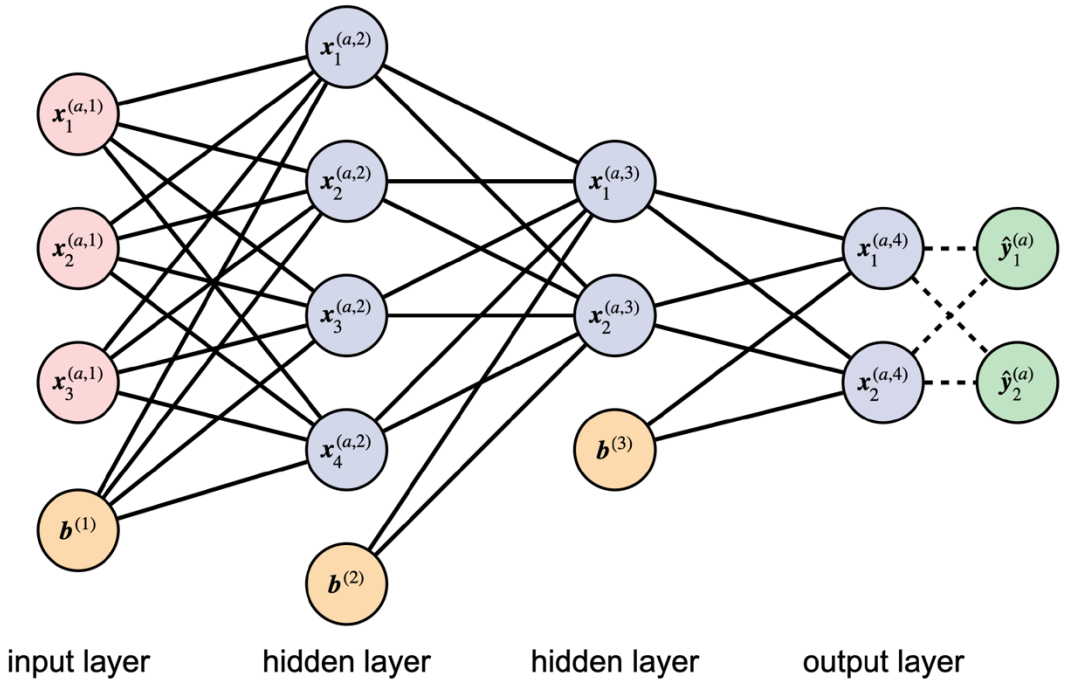
Quantum neural networks

Lesson 3 · Quantum machine learning

Outline

1. Definition
2. Training
3. NN vs QNN
4. Barren plateau
5. Wrap-up

1. Neural network



- Three parts
  - ❑Input layer → hidden layers → output layer
- Each layer
  - ❑$z = Wx + b,$ then $\sigma(z)$
  - ❑Non-linear activation
- Trained
  - ❑W, b

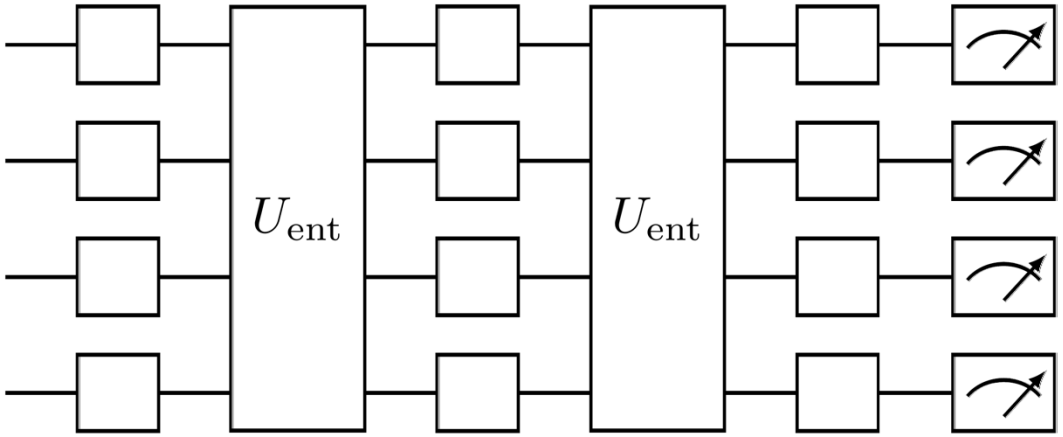
1. Quantum neural network (QNN)

- Circuit: $|0\rangle^{\otimes n}$ → encoding $U(x)$ → ansatz $V(\theta)$ → measure O

1. Encoding: $|\psi(x)\rangle = U(x)|0\rangle^{\otimes n}$ · same as QKM
2. Ansatz $V(\theta)$: trainable circuit · new
3. Measure observable O: e.g. Z on qubit 1

- $f_\theta(x) = \langle\psi(x)|\, V(\theta)^\dagger\, O\, V(\theta)\, |\psi(x)\rangle \in [-1, 1]$
- Prediction: $\mathrm{sign}(f_\theta(x))$

1. Variational block: ansatz $V(\theta)$



- Trainable: single-qubit rotations RX, RY, RZ
  - ❑ Angles = θ
- Fixed: two-qubit gates CNOT / CZ
  - ❑ Entanglement
- Repeat L layers
  - ❑ L is a hyperparameter
- QKM: fixed circuit · trained α outside the quantum computer
- QNN: trained θ inside the circuit

1. QNN = two blocks and a loop

- Train data: (x_i, y_i) → quantum block
- **Quantum**: compute f (hardware: average over shots · simulator: exact)
  1. Encoding: $U(x_i)$
  2. Ansatz: $V(\theta)$, trainable
  3. Measure: $O \rightarrow f_\theta(x_i)$
- **Classical**: train θ
  - ❑Loss: compare $f_\theta(x_i)$ with y_i
  - ❑Optimizer: θ → new θ, back into the ansatz
- Training loop: measure → loss → optimizer → ansatz
- Prediction: new x + trained θ → $f_\theta(x)$ → $\mathrm{sign}(f_\theta(x))$

2. Training: same as NN

	Neural network	QNN
Loss	$L = \frac{1}{2} \Sigma_i (\hat{y}_i - y_i)^2$	$L(\theta) = \frac{1}{2} \Sigma_i (f_\theta(x_i) - y_i)^2$
Update	$W \leftarrow W - \eta \nabla L$	$\theta \leftarrow \theta - \eta \nabla L$ (or Adam)
Gradient	Backprop	$\nabla L = \Sigma_i (f_\theta(x_i) - y_i) \nabla f_\theta(x_i)$
Landscape	Usually non-convex	Usually non-convex (QKM: convex)

New: $\nabla f_\theta(x)$, with θ inside the circuit

2. On hardware, ∇f costs 2 runs per parameter

Quantum hardware

- State cannot be read mid-circuit → no backprop
- $\partial f/\partial\theta_k = ½\,[\,f(\theta_k + \pi/2) - f(\theta_k - \pi/2)\,]$
- 2 circuit runs per parameter, each averaged over shots
- Per training step: 21,840 circuit runs, each × shots
  - ❑ $2PN$ with $P = 24$ parameters, $N = 455$ training points

Simulator (lab)

- State in memory → backprop, as in NN
- $\partial f/\partial\theta_1$ on the lab circuit:
  - ❑ +0.0809 (parameter-shift)
  - ❑ +0.0809 (backprop)
- Same gradient

3. NN vs QNN

	Neural network	QNN
Same recipe as NN		
Model	Layers: affine + activation	Encoding → ansatz $V(\theta)$ → measure O
Optimization	Same loss, GD / Adam, usually non-convex	Identical
Gradient	Backprop, exact	Parameter-shift: 2 runs per parameter, shot noise
What changes		
Features	Learned: hidden layers change with W	Fixed by the encoding; θ cannot change them
Non-linearity in x	At every layer	Only in the encoding; L layers = one unitary
Failure	Vanishing gradient with depth	Barren plateau with qubit number (next)

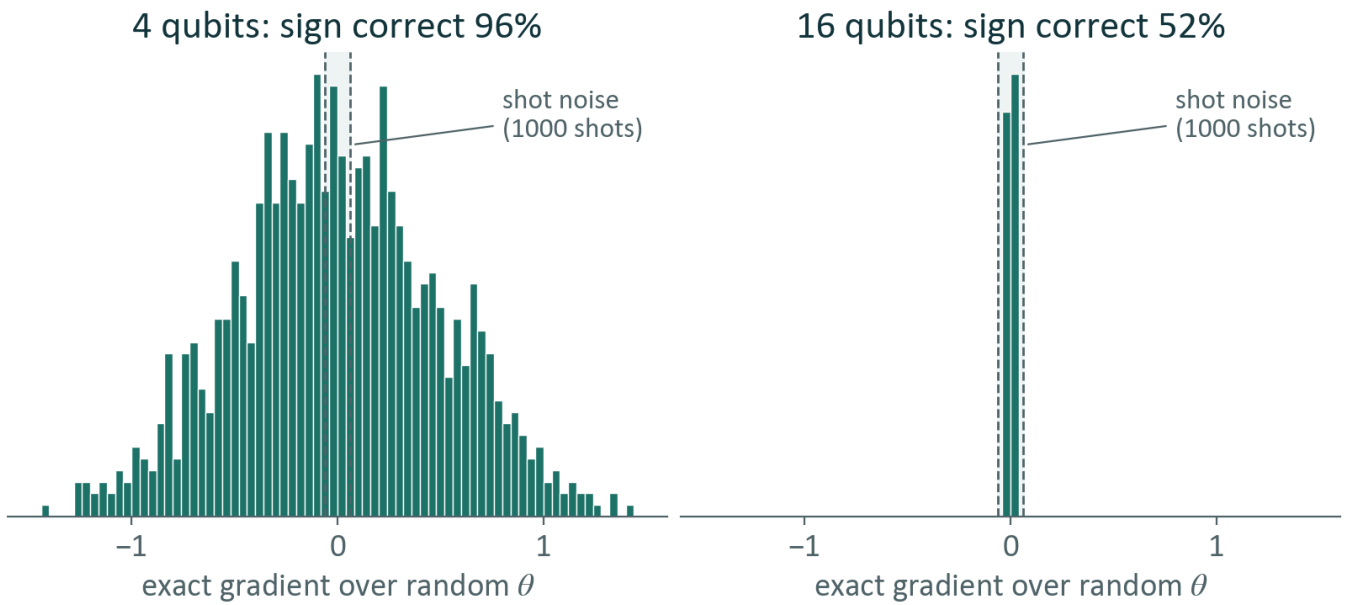
3. QNN as a linear model

- $f_\theta(x) = \langle\psi(x)|\, V(\theta)^\dagger\, O\, V(\theta)\, |\psi(x)\rangle = \langle\psi(x)|\, M(\theta)\, |\psi(x)\rangle$
- x enters only through $|\psi(x)\rangle$ → fixed features
- θ enters only through $M(\theta)$ → trained weights
- Closer to a linear model on fixed features (like logistic regression) than to an NN
- Same features as the quantum kernel: a large class of QML models are kernel methods

4. Barren plateau: gradient below shot noise

NN: vanishing gradient with depth, still computed exactly

1. Random enough ansatz: average $\partial f/\partial\theta = 0,\ \mathrm{Var}[\partial f/\partial\theta] \sim 2^{-n}$
2. Hardware: gradient from shots below shot noise → the sign is a coin flip



Random enough circuit, 1000 random θ, gradient estimated from 1000 shots

4. Three sources of barren plateaus

- Expressive ansatz: deep, random-like circuit
- Global measurement: many qubits measured at once
- Noise: hardware noise, worse with deeper circuits
- Same three as kernel concentration

- Fix: shallow circuit + local measurement
  1. Shallow circuit
  2. Local measurement, e.g. Z on one qubit
  3. Gradient no longer vanishes exponentially

6. Three questions, two models

	QKM	QNN
What can it fit?	Features set by the encoding	Same features Ansatz only picks the weights
Does it train at scale?	Convex: one optimum Fails: concentration	Non-convex: depends on init Fails: barren plateau
Can it run?	$\approx N^2/2$ circuits Simulator: cheap at small N	$\approx 2PN$ per step, many steps Simulator: slower
Strongest	Small N, stable result	Large N (cost $\propto N$) One circuit per prediction

From question to paper

Lesson 4 · Quantum machine learning

Outline

1. Getting started
2. The problem
3. Problem formulation
4. Training
5. Results
6. Limits

1. Our group: QuantumLab

- Mission
  - ❑ Develop new quantum computational methods for molecules and materials, on classical and quantum hardware
- Team (2026)
  - ❑ 1 research associate
  - ❑ 3 PhD students
  - ❑ 4 MSc students
  - ❑ 11 undergraduate researchers
- Infrastructure
  - ❑ HPC cluster COMPHYS (CPU + GPU)
  - ❑ Open-source code pyCMF, Qmes



QuantumLab members, 2026 · University of Science, Ho Chi Minh City

1. How a Project gets started

- Phase 0 · Foundations: textbook, terms and definitions, hands-on software → report
- Advisors: ready → a paper
- Phase 1 · Reproduce: read the paper, re-run its experiments → report
- Then the loop: read → report → experiment → read
  - ❑Each meeting: progress + discussion
- Paper 1 stems from this loop

2. Change the encoding, change the result

Encoding	Acc
RBF (baseline)	0.961
SRx	0.957
HERx	0.963
ZFM	0.922

- New dataset → which encoding?
- Why hard
  - ❑ Expressive circuit → concentration (Lesson 2)
  - ❑ May not match the problem's structure
  - ❑ Number of circuits grows combinatorially
- Today's practice
  - ❑ Fixed circuit
  - ❑ Brute force: $\mathcal{O}(|K|\ N^2)$ circuit runs, per dataset

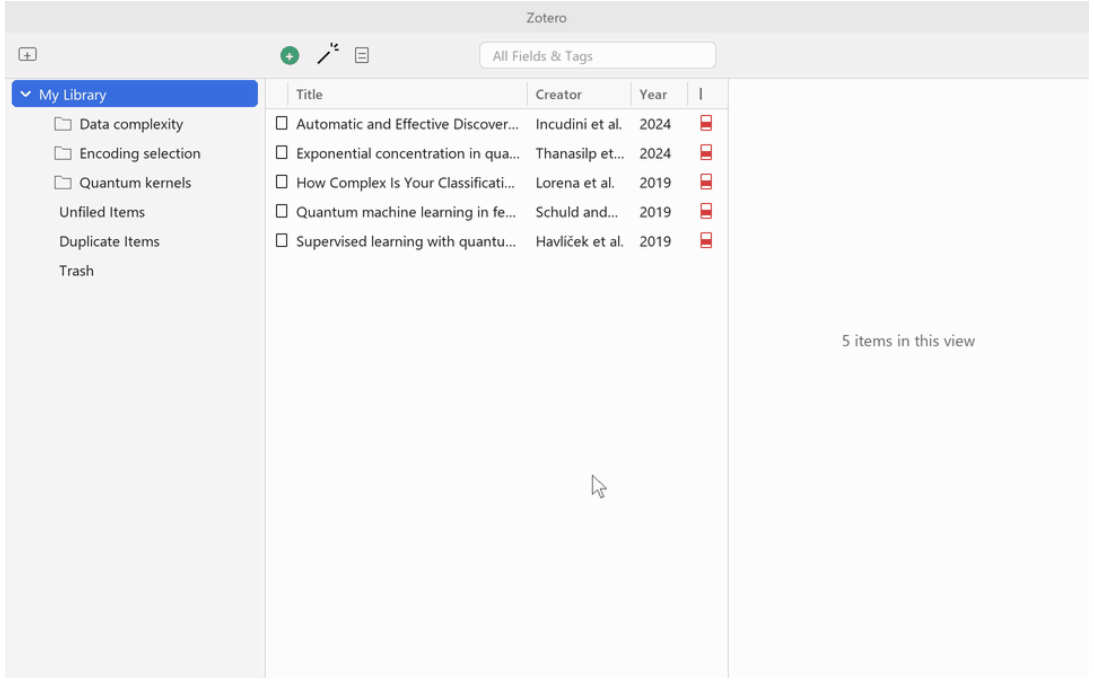
2. Two prior directions, two gaps

	Incudini et al. (2024)	Neto (2025)	Paper 1
Idea	Search over many circuits, scoring each one	Data complexity measures → ML recommends a circuit	Data complexity measures → ML recommends an encoding
Target	Quantum kernel ✓	Variational circuit	Quantum kernel ✓
Learns from the data	✗	✓	✓
New dataset	New optimization loop	One prediction	One prediction
Gap	Costly loop per dataset; ignores the data	Not for quantum kernels	—

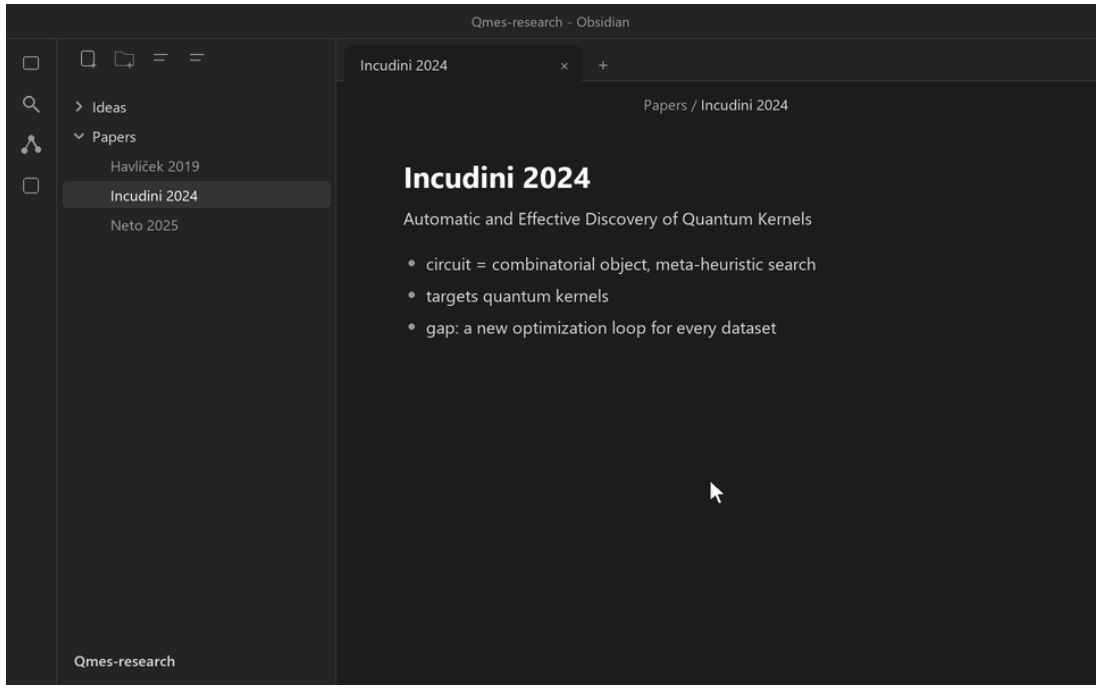
Their gaps are what Paper 1 builds on: quantum kernel + learn from the data

2. Papers in Zotero, ideas in Obsidian

Zotero: collect · organize · cite papers



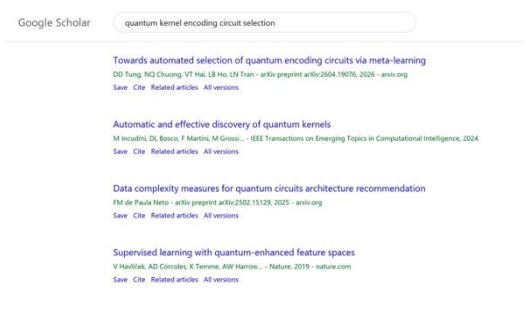
Obsidian: notes · meeting minutes · reports, linked



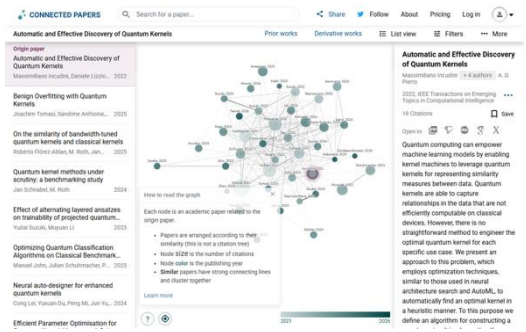
2. Other tools



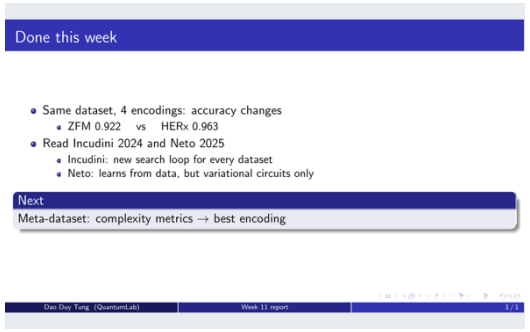
arXiv · read



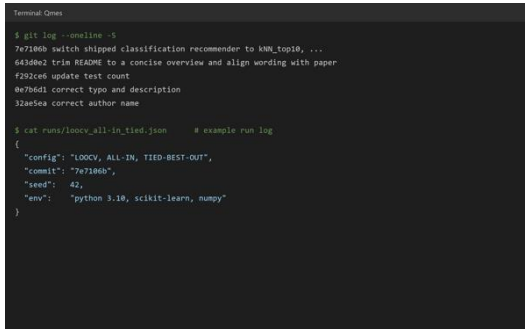
Google Scholar · read



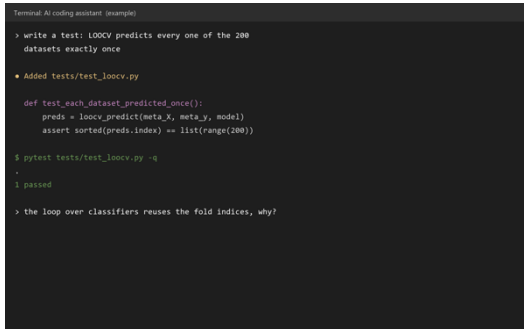
Connected Papers · read



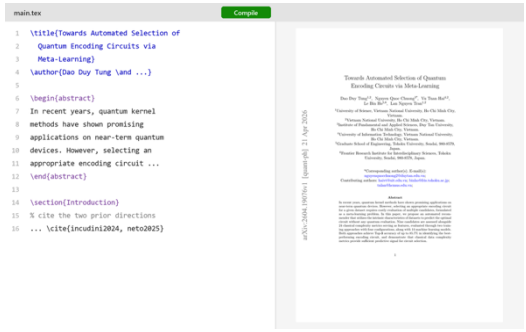
Beamer / Google Slides · report



Git + run log · experiment



AI coding assistant · experiment



LaTeX: Overleaf / local + git · write

Read → report → experiment → write
The loop of slide 4, then the paper

Real: arXiv, Connected Papers, Beamer
Mock-ups: Scholar, terminals, editor

3. A dataset vs a meta-dataset

Normal dataset

x_1	x_2	x_3	x_4	y
0.72	0.92	0.69	2.02	malignant
0.45	1.83	0.67	1.32	benign
1.73	1.02	1.12	2.18	malignant
⋮				

455 rows · 4 features

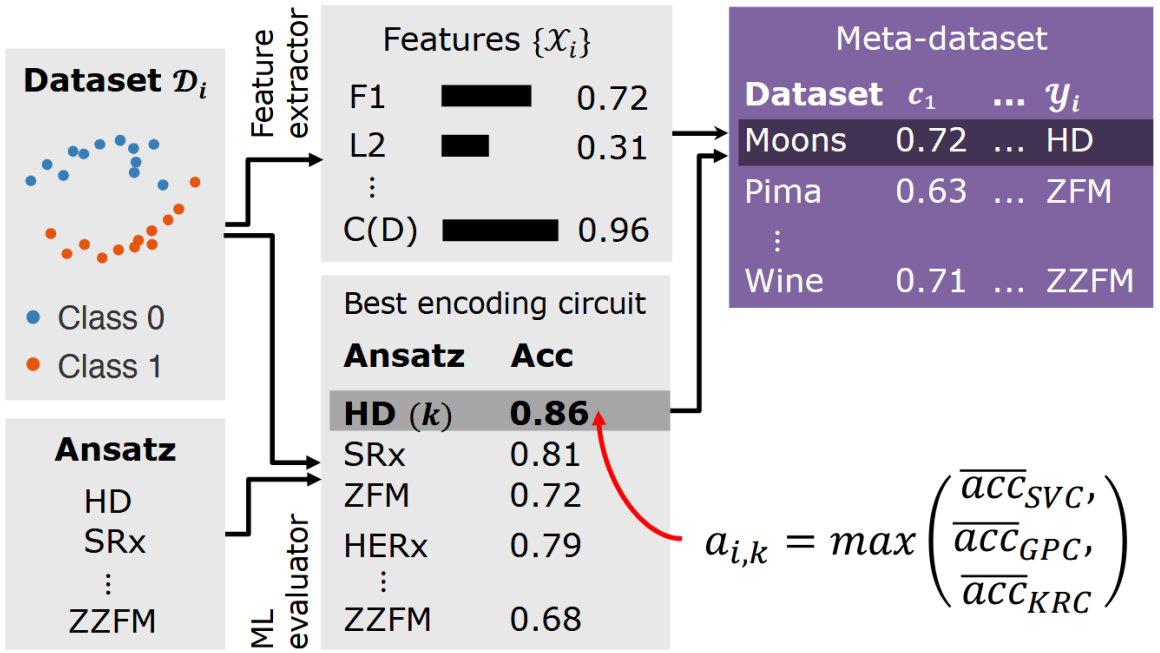
Meta-dataset: one row = one dataset

Dataset	F1	L2	…	Best circuit
Breast cancer	0.17	0.01	…	HERx
Moons	0.27	0.13	…	HD
Pima	0.64	0.22	…	SRx
⋮				

200 rows · 24 metrics

(a) Build the meta-dataset: quantum only here, it generates the labels
(b) Train f
(c) New dataset → Top-k circuits: classical only

3. How to build a row in meta-dataset



① Features
Describe the dataset, classical
L2: error of a linear classifier

② Label
Run every circuit, keep the best
Only quantum step, the costliest

③ Repeat
200 datasets → 200 rows
Most datasets are generated

Source: ***Towards Automated Selection of Quantum Encoding Circuits via Meta-Learning***, Fig. 2

3. How to choose the best circuit?

- Score of circuit on dataset: acc = max(SVC, GPC, KRC)
  - ❑ "Upper bound on what circuit k can do"
- What if the scores are equal, how can we choose the most optimal?
  - ❑ Equal: HD 0.8625 · HERx 0.8625 · SRx 0.8625
  - ❑ Does the single max really show what each circuit can do?
- One winner: label = exactly one circuit, the best
  - ❑ HD is 33% of all labels
- Ties within 1%: label = every circuit within 1% of the best
  - ❑ HD 20% · ZFM 20% · HERx 19% · SRx 17%

4. Building the recommender

- 14 ordinary classifiers (see table)
- MV (majority voting): keeps all 14
  - ❑ Train all 14 on the meta-dataset
  - ❑ Each one votes for a circuit
  - ❑ Most votes = recommendation · vote order = Top-k
- LOOCV: keeps only the best one
  - ❑ Hide one dataset, train on the rest, check the guess
  - ❑ Repeat for every dataset
  - ❑ The classifier that guesses best wins
- A guess counts as right if it is one of the circuits tied for best

Type	Classifiers (14)
Tree-based	Decision tree, random forest
Ensemble	Gradient boosting, AdaBoost, bagging
SVM	Linear, RBF, sigmoid kernel
Neural network	MLP, 1 or 3 hidden layers
Instance-based	k-NN, nearest centroid
Probabilistic	Naive Bayes, logistic regression

4. On the meta-dataset: ties make it learnable

	One winner	Ties within 1%
24 metrics	≈ 0.69	≈ 0.85
1 metric (the best one)	≈ 0.55 – 0.59	—

24 metrics beat 1: no single metric is enough
With ties, the classifier only needs the right family of circuits

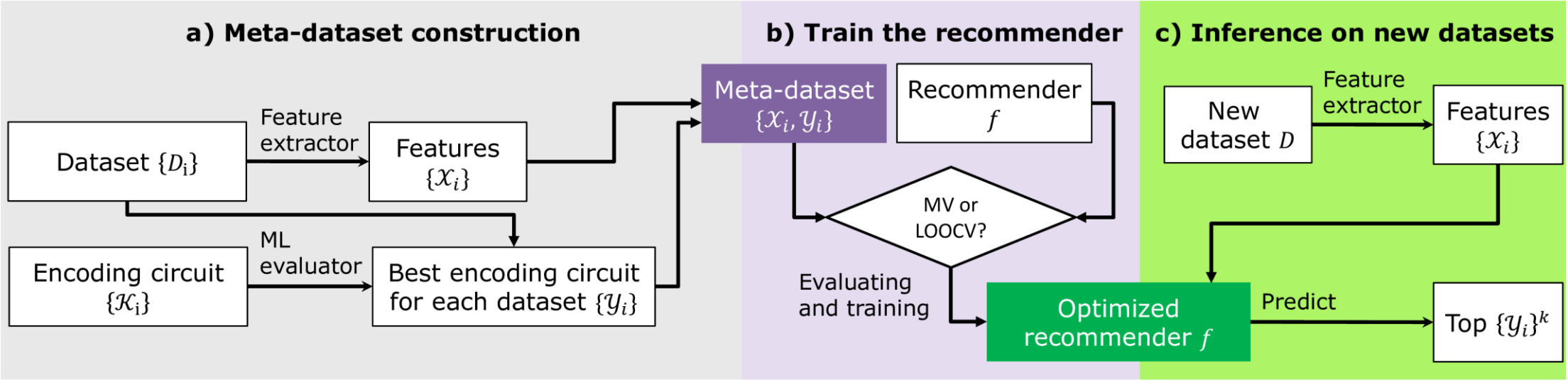
5. Inference on new dataset

Digits 0v1 · Moons · Circles · Synthetic · GaussianQuantiles · MultiRing · Imbalanced

	Top-1	Top-3
MV · 24 metrics	3/7	6/7
LOOCV · 24 metrics (RF)	5/7	6/7
MV · 1 metric (N4)	2/7	5/7
LOOCV · 1 metric (E-GB, N4)	2/7	5/7

Top-3: run ≤ 3 of 9 circuits, not all 9
One metric is not enough

5. Paper 1 in a nutshell



Towards Automated Selection of Quantum Encoding Circuits via Meta-Learning

(a) A point = a dataset, its metrics, its best circuit; quantum runs only here, once, offline
(b) MV or LOOCV: 14 classifiers → one recommender
(c) New dataset → Top-k circuits; no circuit runs

6. Paper 1 answers one question

- Answered
  - ❑ Classical complexity metrics carry enough signal to choose an encoding
  - ❑ New dataset: no circuit runs to get a recommendation
- Limits
  - ❑ Proof-of-concept: 200 datasets, mostly synthetic
  - ❑ Tested on 7 new datasets
  - ❑ Classification only
  - ❑ Research code: no package, no pre-trained model
- Also open: quantum descriptors (expressibility, entanglement) · noise models · recommender → designer
- Lesson 5: from a paper to a package others can use

From paper to package

Lesson 5 · Quantum machine learning

Outline

1. Purpose
2. Bottlenecks
3. Architecture
4. Wrap-up

1. From “is the idea right?” to “can others use it?”

	Paper 1 · research paper	Paper 2 · software paper (Qmes)
Question	Is the idea right?	Can others use it?
What a reader gets	Paper · code on GitHub	Installable package · pre-trained model
To use it on your data	Rerun the pipeline, train f yourself	pip install → recommend(X, y)

Without a recommender: $\mathcal{O}(|K|\ N^2)$ circuit runs per new dataset ($|K|$ circuits · N samples)
With Qmes: a recommendation without running any circuit
We already tried every circuit on ≈ 100 datasets per task, once, offline

We try every circuit once, offline, so users don't have to.

1. Three goals

Usable	Applicable	Extensible
Install and run Recommend without running any circuit	The user's own data Classification + regression Checked on 105 + 86 datasets	Add your own circuit or task Pluggable components

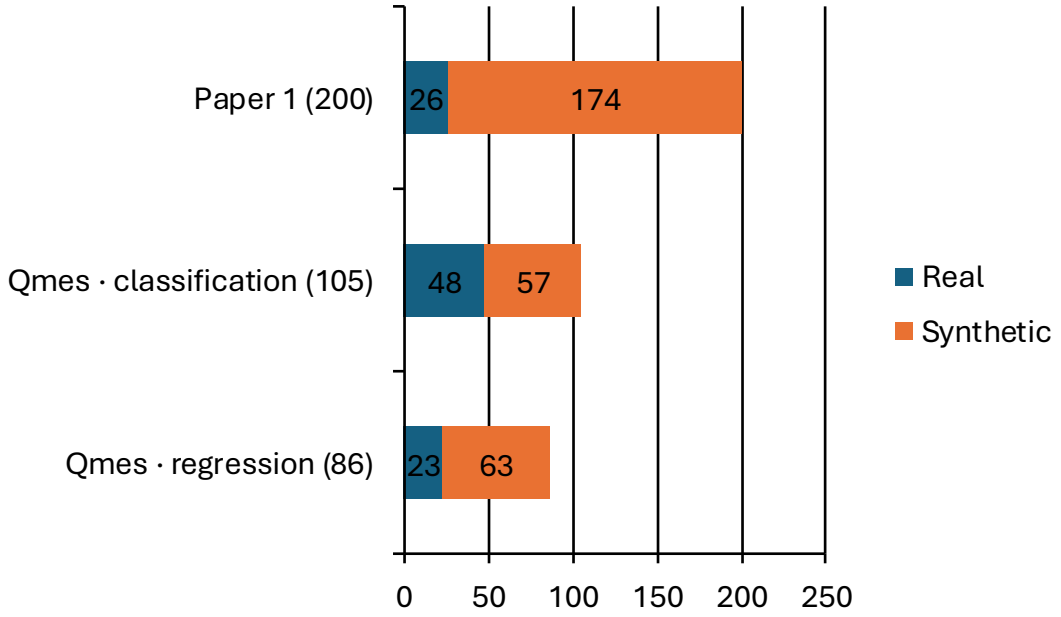
Usable

```
from Qmes import get_extractor, load_default_recommender, recommend
ext = get_extractor("classification")
rec = load_default_recommender("classification")
recommend(X, y, extractor=ext, recommender=rec)
```

No circuit runs, classical only
To use the chosen circuit, you still run it

2. Q1 · From synthetic to real data

Real vs synthetic datasets



- New problems

1. Real data are often imbalanced
   - ❑ Accuracy can look good while the model learns nothing
   - ❑ Score each circuit with MCC
2. Some datasets teach nothing
   - ❑ Every circuit ≈ random, or every circuit ≈ perfect
   - ❑ Drop them before training

2. Q2 · Only ≈100 datasets

- 1 dataset = 1 sample
  label = best of 7 circuits
- ≈100 samples, 7 labels
  → too few examples per label
- Easier question:
  “is A better than B?” (yes/no)

Training (once): a ✓/✗ on every dataset

	D1	D2	D3	…	D100
HD > RY ?	✓	✗	✓	…	✓

New dataset: each answer is a vote

HD > RY ?	✓	**HD +1**
HD > SRx ?	✓	**HD +1**
RY > SRx ?	✗	**SRx +1**

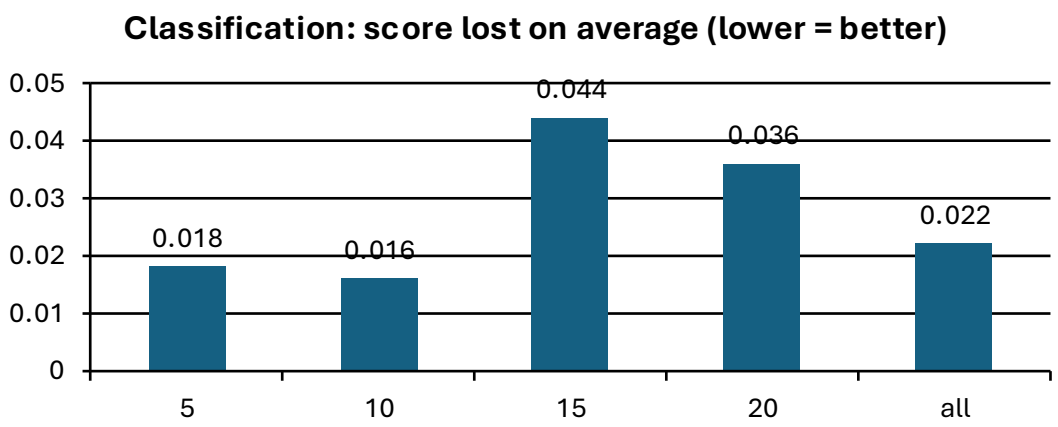
Votes: HD 2 · SRx 1 · RY 0 → Ranking: HD, SRx, RY

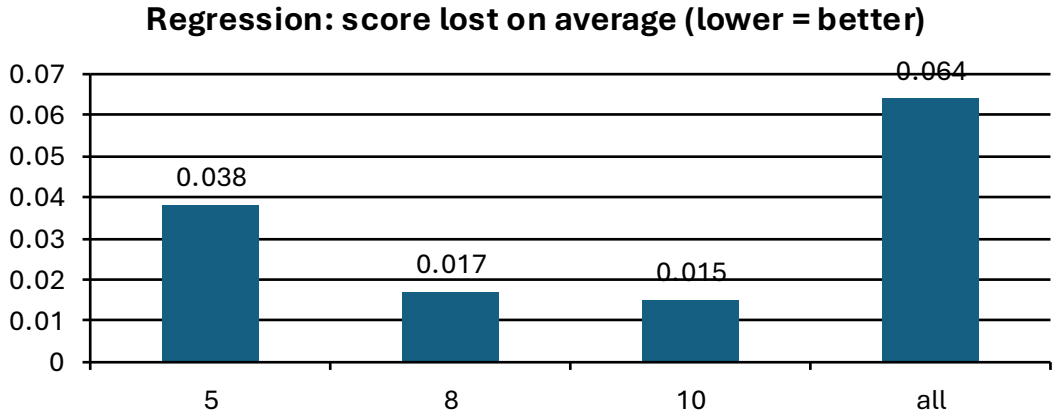
7 circuits → 21 questions, same voting (OvO)

2. Q3 · How many features do we need?

- **Problem:** Paper 1 uses all metrics. Are all of them needed?

- **Our solution:** keep only the most informative metrics
  - ❑rank metrics → keep top k → test each k

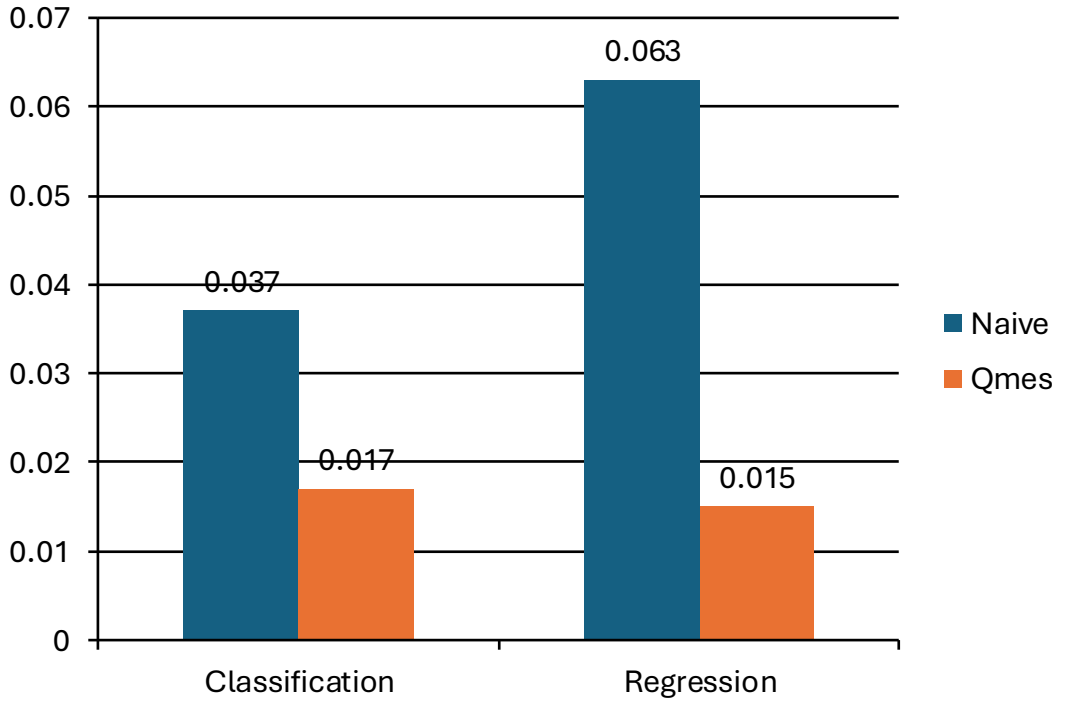




2. Q4 · Performance evaluation metric

Regret = *score of the best circuit − score of the recommended circuit*

- Paper 1: only hit / miss
  - ❑ A miss can lose 0.001 or 0.08 → hit/miss can't tell them apart
- Need: how much is lost? → regret
- With regret: better than a naive choice?
  - ❑ Naive = always the best-on-average circuit
  - ❑ Qmes loses 2–4× less



Qmes: Quantum Meta-Learning for Encoding Selection in Quantum Kernel Methods

2. Q5 · From code to a ready-to-use model

- Code only (Paper 1)
  - ❑ To get a recommendation on your data: rebuild the meta-dataset = run every circuit on every dataset
  - ❑ Then train the recommender yourself
- Qmes package: pip install from GitHub
  - ❑✓ The code
  - ❑✓ The simulator (Qsun): nothing else to install
  - ❑✓ Trained recommenders, one per task

→ Users get the result of the expensive part without redoing it

3. Qmes pipeline

- Offline · run once
  - ❑ Dataset → Extractor: describe the dataset (metrics)
  - ❑ Dataset → Evaluator: run every circuit, score it; the only quantum block (Q1)
  - ❑ Meta-dataset → train the Recommender (Q2 · Q3 · Q4)
  - ❑ Trained Recommender → in the package (Q5)
- Per new dataset · no circuit runs
  - ❑ New dataset → Extractor → Recommender → top-3 circuits
- Each block can be swapped: regression was added this way